

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования

«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»
(МАИ)

ОТЧЁТ
О ЛАБОРАТОРНОЙ РАБОТЕ
по теме:
ПОИСК ФАЙЛОВ ОПРЕДЕЛЕННОГО РАСШИРЕНИЯ

СПИСОК ИСПОЛНИТЕЛЕЙ

Преподаватель

подпись, дата

Павлов О.В.

Студент группы
МЗО-113БВ-25

подпись, дата

Садунян Р.К.

СОДЕРЖАНИЕ

1	Цель и задачи работы	4
2	Описание алгоритма	5
3	Листинг программы с комментариями	6
4	Скриншоты работы программы	10
5	Вывод	11

1 Цель и задачи работы

Цель работы - разработать консольную программу, которая рекурсивно анализирует указанную пользователем папку и находит файлы с заданным расширением.

Задачи работы:

- запросить у пользователя путь к стартовой директории;
- запросить расширение искомых файлов;
- проверить существование указанной папки;
- рекурсивно обойти вложенные каталоги;
- сравнить расширения файлов без учета регистра;
- вывести абсолютные пути найденных файлов и их размер;
- обработать ошибки неверного пути и отсутствия прав доступа;
- вывести итоговое количество найденных файлов.

2 Описание алгоритма

В программе используется рекурсивный обход каталогов средствами библиотеки `std::filesystem`. Пользователь может ввести расширение как с точкой, так и без точки. Перед сравнением расширение нормализуется: к нему добавляется точка, если она отсутствует, а все буквы переводятся в нижний регистр. Расширение каждого найденного файла также переводится в нижний регистр, поэтому файлы `.cpp`, `.CPP` и `.CpP` считаются совпадающими.

Псевдокод основного алгоритма:

- ввести путь к папке
- ввести расширение файла
- нормализовать расширение

- если путь пустой или не существует:
 - вывести ошибку и завершить программу

- если путь не является папкой:
 - вывести ошибку и завершить программу

- создать пустой список найденных файлов

- процедура Поиск(папка):
 - попытаться открыть папку
 - если открыть папку нельзя:
 - вывести предупреждение и вернуться

- для каждого элемента в папке:
 - если элемент является папкой:
 - Поиск(элемент)
 - иначе если элемент является обычным файлом:
 - если расширение элемента совпадает с искомым:
 - добавить абсолютный путь и размер файла в список

- отсортировать список найденных файлов по пути
- вывести найденные файлы
- вывести количество найденных файлов

3 Листинг программы с комментариями

Ниже приведён полный исходный код программы.

```
#include <algorithm>
#include <cctype>
#include <locale>
#include <filesystem>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <system_error>
#include <vector>

using namespace std;
namespace fs = std::filesystem;

struct FoundFile {
    fs::path path;
    uintmax_t size = 0;
    bool sizeAvailable = false;
};

string trim(const string& value) {
    const size_t first = value.find_first_not_of(" \t\r\n");
    if (first == string::npos) {
        return "";
    }

    const size_t last = value.find_last_not_of(" \t\r\n");
    return value.substr(first, last - first + 1);
}

string toLower(string value) {
    transform(value.begin(), value.end(), value.begin(), [](unsigned char symbol) {
        return static_cast<char>(tolower(symbol));
    });
    return value;
}

string normalizeExtension(const string& input) {
    string extension = trim(input);

    if (!extension.empty() && extension.front() != '.') {
        extension = "." + extension;
    }

    return toLower(extension);
}

fs::path makeAbsolutePath(const fs::path& path) {
    error_code error;
    fs::path result = fs::weakly_canonical(path, error);
    if (!error) {
        return result;
    }

    error.clear();
    result = fs::absolute(path, error);
    if (!error) {
        return result.lexically_normal();
    }

    return path;
}
```

```

}

string formatSize(uintmax_t bytes) {
    ostringstream output;
    output << fixed << setprecision(2) << static_cast<double>(bytes) / 1024.0 << " KB";
    return output.str();
}

string fileWord(size_t count) {
    const size_t lastTwoDigits = count % 100;
    const size_t lastDigit = count % 10;

    if (lastTwoDigits >= 11 && lastTwoDigits <= 14) {
        return "файлов";
    }
    if (lastDigit == 1) {
        return "файл";
    }
    if (lastDigit >= 2 && lastDigit <= 4) {
        return "файла";
    }
    return "файлов";
}

void addFileIfExtensionMatches(
    const fs::directory_entry& entry,
    const string& targetExtension,
    vector<FoundFile>& foundFiles) {
    const string currentExtension = toLower(entry.path().extension().string());
    if (currentExtension != targetExtension) {
        return;
    }

    error_code error;
    const uintmax_t size = fs::file_size(entry.path(), error);

    foundFiles.push_back({
        makeAbsolutePath(entry.path()),
        size,
        !error
    });
}

void searchFiles(
    const fs::path& directory,
    const string& targetExtension,
    vector<FoundFile>& foundFiles) {
    error_code error;
    fs::directory_iterator iterator(directory, error);

    if (error) {
        cerr << "Предупреждение: нет доступа к \""
              << makeAbsolutePath(directory).string()
              << "\": " << error.message() << '\n';
        return;
    }

    for (const fs::directory_entry& entry : iterator) {
        error_code statusError;
        const fs::file_status status = entry.symlink_status(statusError);

        if (statusError) {
            cerr << "Предупреждение: не удалось прочитать \""
                  << makeAbsolutePath(entry.path()).string()
                  << "\": " << statusError.message() << '\n';
            continue;
        }
    }
}

```

```

        // Символические ссылки не раскрываются, чтобы избежать циклов обхода.
        if (fs::is_directory(status)) {
            searchFiles(entry.path(), targetExtension, foundFiles);
        } else if (fs::is_regular_file(status)) {
            addFileIfExtensionMatches(entry, targetExtension, foundFiles);
        }
    }
}

int main() {
    setlocale(LC_ALL, "");

    string directoryInput;
    string extensionInput;

    cout << "Введите путь для поиска: ";
    getline(cin, directoryInput);

    cout << "Введите расширение (например, cpp): ";
    getline(cin, extensionInput);

    const fs::path startDirectory = trim(directoryInput);
    const string targetExtension = normalizeExtension(extensionInput);

    if (startDirectory.empty()) {
        cerr << "Ошибка: путь к папке не может быть пустым.\n";
        return 1;
    }

    if (targetExtension.empty() || targetExtension == ".") {
        cerr << "Ошибка: расширение файла не может быть пустым.\n";
        return 1;
    }

    error_code error;
    if (!fs::exists(startDirectory, error) || error) {
        cerr << "Ошибка: указанный путь не существует.\n";
        return 1;
    }

    if (!fs::is_directory(startDirectory, error) || error) {
        cerr << "Ошибка: указанный путь не является папкой.\n";
        return 1;
    }

    cout << "Поиск файлов с расширением " << targetExtension << " ...\n";

    vector<FoundFile> foundFiles;
    searchFiles(startDirectory, targetExtension, foundFiles);

    sort(foundFiles.begin(), foundFiles.end(), [](const FoundFile& left, const FoundFile& right) {
        return left.path.string() < right.path.string();
    });

    cout << "Найденные файлы:\n";
    if (foundFiles.empty()) {
        cout << "Файлы не найдены.\n";
    } else {
        for (const FoundFile& file : foundFiles) {
            cout << file.path.string() << " (";
            if (file.sizeAvailable) {
                cout << formatSize(file.size);
            } else {
                cout << "размер недоступен";
            }
            cout << ")\n";
        }
    }
}

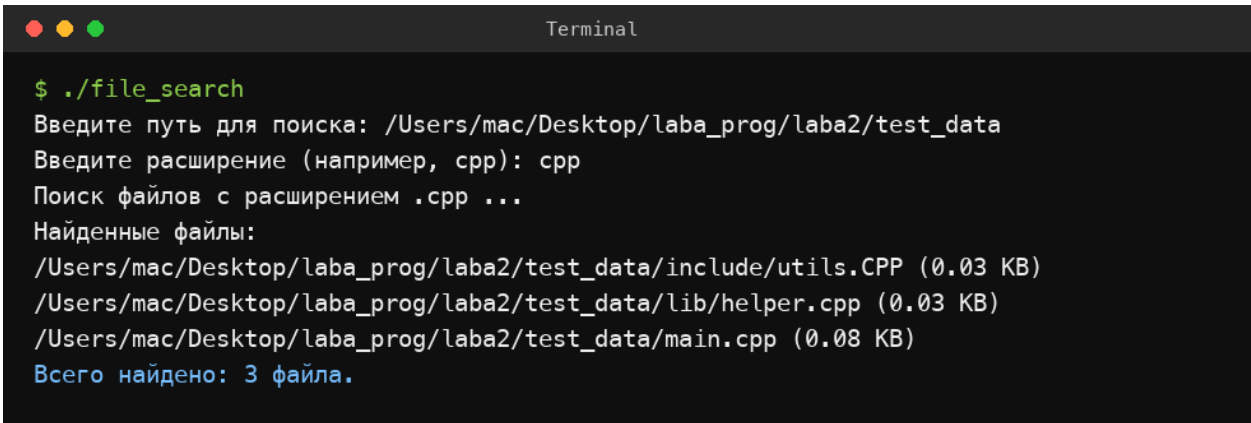
```



```
    }  
}  
  
cout << "Всего найдено: " << foundFiles.size()  
      << ' ' << fileWord(foundFiles.size()) << ".\n";  
  
return 0;  
}
```

4 Скриншоты работы программы

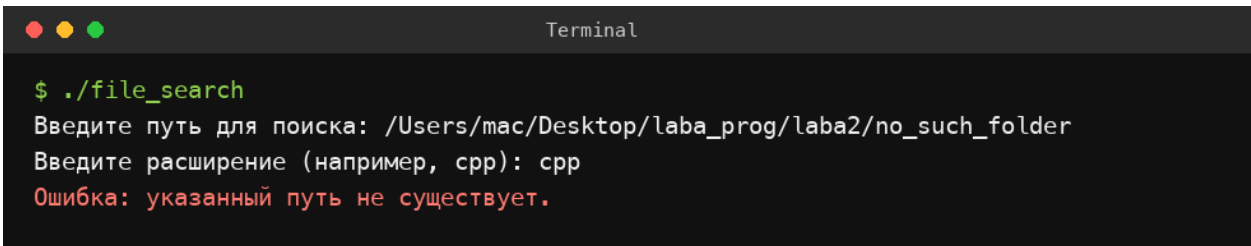
Для проверки был создан каталог `test_data`, содержащий файлы `main.cpp`, `helper.cpp`, `utils.CPP`, а также файлы с другими расширениями. На первом примере видно, что программа находит файлы `.cpp` независимо от регистра расширения и выводит абсолютные пути.



```
Terminal
$ ./file_search
Введите путь для поиска: /Users/mac/Desktop/laba_prog/laba2/test_data
Введите расширение (например, cpp): cpp
Поиск файлов с расширением .cpp ...
Найденные файлы:
/Users/mac/Desktop/laba_prog/laba2/test_data/include/utils.CPP (0.03 KB)
/Users/mac/Desktop/laba_prog/laba2/test_data/lib/helper.cpp (0.03 KB)
/Users/mac/Desktop/laba_prog/laba2/test_data/main.cpp (0.08 KB)
Всего найдено: 3 файла.
```

Рисунок 1 — Поиск файлов с расширением `cpp`

На втором примере показана обработка неверного пути.



```
Terminal
$ ./file_search
Введите путь для поиска: /Users/mac/Desktop/laba_prog/laba2/no_such_folder
Введите расширение (например, cpp): cpp
Ошибка: указанный путь не существует.
```

Рисунок 2 — Обработка ошибки неверного пути

5 Вывод

В ходе лабораторной работы была разработана консольная программа для поиска файлов по расширению в указанной директории. Были использованы возможности стандартной библиотеки C++17 `std::filesystem`: проверка существования пути, определение типа файловой системы, получение размера файла и обход каталогов.

Программа корректно обрабатывает ввод расширения с точкой и без точки, сравнивает расширения без учета регистра и выводит найденные файлы с абсолютными путями. Также предусмотрена обработка ошибок: пустой путь, несуществующая директория, путь к файлу вместо папки и отсутствие доступа к отдельным каталогам.